Bielefeld Augmented Reality Tracker (BART) - Manual

March 22, 2015

Thies Pfeiffer and Patrick Renner

CITEC - Center of Excellence Cognitive Interaction Technology
Inspiration 1
33615 Bielefeld www: http://www.cit-ec.de
Contact: Thies.Pfeiffer@Uni-Bielefeld.de

Contents

List of Figures

List of Tables

1 Introduction

This is BART.

2 Installation

2.1 Where to get BART?

At the time being, BART is hosted as an internal project at CITEC. In the near future a release as OpenSource library is planned. Until then, please use the contact information provided at the beginning of this document to get access to the most recent version of BART.

2.2 Requirements

2.2.1 Supported Platforms

BART is based on open source libraries and standard C++ which are available on many platforms. So in principle, BART should be able to run on any platforms supporting all required libraries. At CITEC, we have used and tested BART on the following platforms:

- Linux
  - Ubuntu 14.04 LTS (64bit)
- Microsoft Windows
  - Version 8 (32bit and 64bit)

2.3 Compiling BART

The following instructions are only relevant when you have retrieved the source code for BART and want to compile it on your own.

Instructions to follow...

3 First Steps

This chapter presents a minimal example on how to use BART in the most simplest way.

4 Developer Instructions

This chapter provides more detailed instructions on how to use BART in your own application. It also provides step-by-step examples.

4.1 Example: Integrating BART with RSB

4.2 Example: Using BART with InstantIO

5 Inside BART

This chapter will provide a deeper insight in how BART functions.

5.1 General Architecture

5.2 Detecting Fiducial Markers

5.3 Tracking

6 Evaluation

This chapter will provides an evaluation of BART.

Bibliography

Appendix

Appendix

A.1 Calibrating the Camera

Images retrieved from a camera might be distorted by the lenses the cameras are using. This can be easily detected when taking a picture of a straight line or even better a chess board. If the lines that are straight in reality are bending in the image provided by the camera, then the camera image is affected by distortion.

Happily, in most cases the image can be corrected (undistorted) by applying computer vision techniques. BART can also undistort the camera image, but for this, it requires some information from the camera. The camera needs to be calibrated and a special file with information about the distortion can be supplied to BART.

To create this calibration file, a special tool called **cameraCalibration** is provided together with BART. This tool can either take a collection of files taken with a camera, or directly use a camera connected to the computer to do the calibration.

A.1.1 Calibration Procedure

1. Create a configuration file **CHOOSENAME.yaml** for the calibration.
2. A minimal configuration file should specify the elements shown in Listing A.1.
3. Print the calibration pattern, e.g. the chessboard pattern.
4. Run **cameraCalibration *CHOOSENNAME.yaml*** to start the calibration procedure. Follow the instructions given on the console or the GUI. If everything works as desired, the calibration file specified in the configuration should have been created.

A.2 Testing BART with testMarkerTracker

A.2.1 Running testMarkerTracker

The program testMarkerTracker comes with a couple of command line options. In addition to that, the behavior of the tracking can be configured in a special configuration file.

If you want, for example, to analyze a previously recorded video, you can call testMarkerTracker as follows: **testMarkerTracker.exe –file=myvideo.avi –config tracking.yaml**.

More options are provided by adding the parameter –help: **testMarkerTracker –help**.

```
<?xml version="1.0"?>
<opencv_storage>
 <Settings>
  <!-- Number of inner corners per item row and column. -->
  <BoardSize_Width>5</BoardSize_Width>
  <BoardSize_Height>8</BoardSize_Height>

  <!-- The resolution to be chosen (if possible) for the camera.
         Should be the same as the resolution later used in the
         program at runtime. -->
  <Camera_Width>1280</Camera_Width>
  <Camera_Height>960</Camera_Height>

  <!-- The size of a square of the calibration board in some user
        defined metric system (pixel, millimeter). This defines
        the dimensions later provided by the tracking. If undecided,
        measure the squares in terms of millimeters. -->
  <Square_Size>30</Square_Size>

  <!-- The type of input used for camera calibration.
         One of:
           CHESSBOARD
           CIRCLES_GRID
           ASYMMETRIC_CIRCLES_GRID -->
  <Calibrate_Pattern>"CHESSBOARD"</Calibrate_Pattern>

  <!-- The input to use for calibration.
      To use an input camera
          -> give the ID of the camera, like "1"
      To use an input video
          -> give the path of the input video, like "/tmp/x.avi"
      To use an image list
          -> give the path to the XML or YAML file containing
              the list of the images, like "/tmp/circles_list.xml"
      -->
  <Input>"0"</Input>

  <!--  If true (non-zero) we flip the input images
         around the horizontal axis.-->
  <Input_FlipAroundHorizontalAxis>
     0</Input_FlipAroundHorizontalAxis>

  <!-- Time delay between frames in case of camera. -->
  <Input_Delay>500</Input_Delay>

  <!-- How many frames to use, for calibration. -->
  <Calibrate_NrOfFrameToUse>80</Calibrate_NrOfFrameToUse>

  <!-- ADD EXPERT SETTINGS HERE IF NEEDED -->
 </Settings>
</opencv_storage>
```

Listing A.1 *Configuration file example for camera calibration - Most important settings*

```
<!-- Consider only fy as a free parameter,
       the ratio fx/fy stays the same
       as in the input cameraMatrix. Use or not setting.
       0 - False
       Non-Zero - True -->
<Calibrate_FixAspectRatio> 1 </Calibrate_FixAspectRatio>

<!-- If true (non-zero) tangential distortion coefficients
      are set to zeros and stay zero. -->
<Calibrate_AssumeZeroTangentialDistortion>
    0</Calibrate_AssumeZeroTangentialDistortion>

<!-- If true (non-zero) the principal point is not changed
      during the global optimization. -->
<Calibrate_FixPrincipalPointAtTheCenter>
    0</Calibrate_FixPrincipalPointAtTheCenter>

<!-- The name of the output log file. -->
<Write_outputFileName>"camera_MYNAME.xml"</Write_outputFileName>

<!-- If true (non-zero) we write to the output file
      the feature points.-->
<Write_DetectedFeaturePoints>1</Write_DetectedFeaturePoints>

<!-- If true (non-zero) we write to the output file
      the extrinsic camera parameters.-->
<Write_extrinsicParameters>1</Write_extrinsicParameters>

<!-- If true (non-zero) we show after calibration the
      undistorted images.-->
<Show_UndistortedImage>1</Show_UndistortedImage>
```

Listing A.2 *Configuration file example for camera calibration - Expert settings*